
```

int longestSingle1(int [][] A, int n)
    boolean [] S = new boolean[1...n] = {false}
    int maxSoFar = 0
    int start = 1
    int end = 1
    while end ≤ n do
        if S[A[end]] then                                     % Found duplicate
            while A[start] ≠ A[end] do                         % Remove elements before duplicate
                S[A[start]] = false
                start = start + 1
            start = start + 1                                     % Start after duplicate
        else
            S[A[end]] = true                                     % Add element
            maxSoFar = max(maxSoFar, end - start + 1)         % Update maximum, if needed
            end = end + 1
    return maxSoFar

```

La complessità è uguale a quella precedente, i fattori moltiplicativi e l'occupazione di memoria leggermente inferiori.

Esercizio B1

Questo esercizio è simile ad un esercizio che era stato dato in passato. Utilizziamo programmazione dinamica. Sia $DP[i][r]$ il numero di modi con cui è possibile ottenere il valore r utilizzando i primi i valori.

$DP[i][r]$ può essere calcolato nel modo seguente:

$$DP[i][r] = \begin{cases} 1 & r = 0 \\ 0 & r > 0 \wedge i = 0 \\ DP[i-1][r] + DP[i-1][r - A[i]] & i > 0 \wedge r > 0 \wedge A[i] \leq r \\ DP[i-1][r] & i > 0 \wedge r > 0 \wedge A[i] > r \end{cases}$$

- Se $r = 0$, esiste un modo per ottenere il valore zero - non selezionare alcun elemento
- Se $r > 0$ e $i = 0$, ho esaurito i valori e non ho ancora raggiunto il valore 0
- Altrimenti, se $A[i] \geq r$ ho due possibilità, prendere e non prendere il valore. Se invece $A[i] < r$, devo ignorarlo.

Utilizziamo memoization per tradurre l'equazione ricorsiva in codice:

```

int countMenu(int[] A, int n, int k)
    int[] DP = new int[1...n][1...k] = {-1}
    return countRec(A, n, k, DP)

```

```

int countRec(int[] A, int i, int r, int[] DP)
{
    if r == 0 then
        return 1
    else if i == 0 then
        return 0
    else
        if DP[i][r] < 0 then
            DP[i][r] = countRec(A, i - 1, r, DP)
            if A[i] ≤ r then
                DP[i][r] = DP[i][r] + countRec(A, i - 1, r - A[i], DP)
            return DP[i][r]
        return DP[i][r]
}

```

La complessità è limitata superiormente dalla dimensione della matrice: $O(nr)$, pseudopolinomiale nella dimensione dell'input.

Si noti che se più piatti hanno lo stesso costo, verranno contati separatamente (come è giusto che sia). Un'estensione dell'algoritmo che mostra i menù possibili dovrebbe ritornare ognuna di queste possibilità. Si noti inoltre che per come è organizzato l'algoritmo, le scelte vengono effettuate nell'ordine inverso in cui sono memorizzati i piatti, e quindi non ci sono problemi di permutazioni.

Esercizio B2

Sia $DP[i][j]$ la lunghezza della più lunga sottostringa comune che termina in posizione i nella stringa P e nella posizione j nella stringa T . Il valore può essere calcolato tramite questa formula ricorsiva:

$$DP[i][j] = \begin{cases} 0 & i = 0 \vee j = 0 \\ 0 & i > 0 \wedge j > 0 \wedge P[i] \neq T[j] \\ DP[i-1][j-1] + 1 & i > 0 \wedge j > 0 \wedge P[i] = T[j] \end{cases}$$

Si noti che al termine bisognerà cercare il massimo sull'intera matrice, non nella sola casella (m, n) .

```

int LCSubstring(ITEM[] P, ITEM[] T, int n, int m)
{
    int[][] DP = new int[0...n][0...m] = {0}
    int maxSoFar = 0
    for i = 1 to n do
        for j = 1 to m do
            if P[i] == T[j] then
                DP[i][j] = DP[i-1][j-1] + 1
                maxSoFar = max(maxSoFar, DP[i][j])
            else
                DP[i][j] = 0
    return maxSoFar
}

```

Dovendo calcolare l'intera matrice, il costo computazionale è pari a $O(mn)$.

Informazioni aggiuntive Questo problema può essere risolto in tempo $O(m+n)$ utilizzando una struttura dati particolare, chiamata *generalized suffix tree*.

https://en.wikipedia.org/wiki/Longest_common_substring_problem

Esercizio B3

Si utilizza la tecnica backtrack utilizzando uno stack. In questa versione, partiamo dal fondo. Se l'elemento che si sta considerando è più piccolo del valore che stiamo considerando, abbiamo due possibilità: si può inserire oppure no nello stack. Se invece è più grande, viene saltato.

Nel caso di un vettore ordinato, ognuna delle 2^n sottosequenze deve essere stampata, con costo $\Theta(n)$. La complessità computazionale è quindi $O(n \cdot 2^n)$.

```
printIncreasing(int[] A, int n)
```

```
    STACK S = Stack()
```

```
    printRec(A,n,S)
```

```
printRec(int[] A, int i, STACK S)
```

```
    if i == 0 then
```

```
        | print S
```

```
% LIFO order
```

```
    else
```

```
        if S.isEmpty() or A[i] < S.top() then
```

```
            | S.push(A[i])
```

```
            | printRec(A, i - 1, S)
```

```
            | S.pop()
```

```
        printRec(A, i - 1, S)
```
